Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Mediator (Arabulucu) kalıbı (ekrandan, 00:10:00, 00:12:30)
  - Amaç: birbiriyle eşten eşe (peer-to-peer) haberleşen bir nesne kümesindeki karmaşık ilişkiyi merkezi bir noktada toplamak. Tam bağlı n düğümde yaklaşık n^2 bağlantı olur; her nesne çok sayıda başka nesneyle ilişkili olur.
  - Örnek: font seçme penceresi. Liste, metin kutusu, kalınlık, italik, punto gibi GUI bileşenlerinde yapılan her değişiklik önizleme alanına anında yansıtılır. Her bileşenin önizleme alanına doğrudan müdahale etmesi hem karmaşıklık hem kopyala-yapıştır kod üretir.
  - Örnek çözüm: GUI nesnelerine ortak bir arayüz (`Widget`) hazırlanır; aralarındaki etkileşim başka bir soyut sınıfta (`DialogDirector`) tanımlanır. Slayta göre bu soyut sınıftan gerçeklemeler Template Method kalıbıyla oluşturulur.

```
DialogDirector {ShowDialog(); CreateWidgets() «soyut»; WidgetChanged(Widget) «soyut»}
DialogDirector ◁── FontDialogDirector {CreateWidgets(); WidgetChanged(Widget)}
Widget ──director──► DialogDirector
Widget {Changed()}    not: Changed() içinde director->WidgetChanged(this)
Widget ◁── ListBox {GetSelection()}
Widget ◁── EntryField {SetText()}
FontDialogDirector ──list──► ListBox
FontDialogDirector ──field──► EntryField
```

  - Kalıp yapısı: soyut `Colleague` (meslektaş, eş) bir `Mediator` referansı tutar; `ConcreteColleague1`, `ConcreteColleague2` mesajlarını birbirlerine değil `ConcreteMediator` üzerinden gönderir; `ConcreteMediator` da somut meslektaşlara referans tutar.

```
Colleague ──mediator──► Mediator
Mediator ◁── ConcreteMediator
Colleague ◁── ConcreteColleague1, ConcreteColleague2
ConcreteMediator ──► ConcreteColleague1
ConcreteMediator ──► ConcreteColleague2
```

  - Mediator'ın bir meslektaşa referansı yoksa o meslektaşa mesaj iletilemez; tüm meslektaşlara referans varsa herkes herkese Mediator üzerinden mesaj gönderebilir. Belirli meslektaş türleri arasındaki iletişim için farklı somut Mediator alt sınıfları da kurulabilir.
- Mediator ile Observer farkı
  - Widget'tan Mediator'a "değiştim" bildirimi kısmı sınıf şeması olarak Observer'a benzer. Ama amaç değişiklikten haberdar etmek değildir; eşten eşe kurulacak $m \times n$ ilişkiyi merkezi bir noktada toplamaktır. Mediator'da tek bir arabulucu nesne vardır ve mesaj eşten arabulucuya, arabulucudan ilgili diğer eşe gider.
- Mediator kalıbının eleştirisi
  - Karmaşıklık meslektaşlardan arabulucuya taşınır; biraz daha kolay yönetilir ama yine karmaşıktır, sıfıra inmez.
  - Meslektaşlar arasındaki mesajlaşma protokolü iyi tanımlıysa ve sık değişmeyecekse kalıp kurulabilir; etkileşimler iyi tanımlı değilse ve sürekli değişecekse kurulmaması daha iyidir.
- Refactoring (yeniden yapılandırma) nedir? (ekrandan, 00:20:00)
  - Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmektir: dışarıdan görünen davranış (dış arayüz, metot imzaları, verilen hizmetler) korunarak iç

yapı değiştirilir. Tersine mühendislikle karıştırılmamalıdır.
  - Amaç kodun iç kalite özelliklerini iyileştirmektir: anlaşılabilirlik, esneklik, yeniden kullanılabilirlik, yüksek uyum ve düşük bağımlılık. Bakım masrafını azaltan her değişiklik bu kapsamdadır.
  - Gereksinimler eklendikçe yazılım "yamalı bohçaya" döner; refactoring tasarımı daha iyi bir yöne götürür.
  - Önerilen yol: küçük adımlarla, artımsal ve çevik biçimde yapmak. Büyük bir yeniden tasarım uzun sürer ve o sürede gelen yeni gereksinimler yeni tasarımı da yetersiz bırakabilir.
  - Her değişiklikten sonra sınama yapılmalıdır: en azından değişikliğin etkilediği ve onun bir adım ötesindeki varlıkları kapsayan birim testleri yeniden çalıştırılır. Refactoring ve birim sınaması birbirini izleyen bir sarmal biçiminde yürütülür.
- Kod kusuru (code smell)
  - Kodun "kötü kokmaya" başlaması: yamalı bohçaya dönmesi, yeni değişiklikleri kaldıracak esnekliği kaybetmesi. Hoca "kusurlu kod / kod kusuru" demeyi tercih ediyor ve bunun hata (bug) olmadığını vurguluyor.
  - Kusurları erken fark etmek için adlarını ve nedenlerini bilmek gerekir; birikirse kolay düzeltilebilecek noktayı geçeriz. Bir refactoring adımı da yeni kusurlar doğurabilir.
- Birim sınaması (unit testing) ve JUnit (ekrandan, 00:30:00)
  - En küçük yazılım bileşeninin sınanmasıdır; nesneye dayalı programlamada bireysel sınıfların sınanmasıdır.
  - Ne zaman tasarlanır: kodlamadan önce (TDD, Test Driven Design), kodlama sırasında ya da ardından. Bir sınıfın ihtiyaç duyduğu ama henüz yazılmamış sınıfların yerine vekil (stub, dummy, surrogate, proxy) sınıflar kullanılır; örneğin her zaman "başarılı" döndüren bir metot. Vekil, gerçek sınıf tamamlanana dek kullanılır.
  - Hocanın görüşü: bir şeyi test etmekte zorlanıyorsanız büyük olasılıkla doğru tasarlamamışsınızdır. Tüm kod bitince test yazmak geç kalmaktır; "biraz kod, biraz test" önerilir.
  - JUnit, Java için birim sınama çerçevesidir (xUnit ailesi; C# için csUnit). Eclipse ve NetBeans ile hazır gelir.
- JUnit 3.x yapısı
  - Her uygulama sınıfı (`AppClass1`) için adı `Test` ile başlayan bir test sınıfı (`TestAppClass1`) yazılır; bu sınıf `TestCase`'ten kalıtımla türetilir. Her `testXxx` metodu bir test durumudur (test case); metot adları `test` ile başlamak zorundadır.
  - `setUp` metodu sahneyi hazırlar: her test metodundan önce otomatik çalışır, nesneleri ve ilişkileri kurar.
  - Test sınıfları bir `suite` içinde birleştirilip tek adımda çalıştırılabilir; suite de adı `Test` ile başlayan bir sınıfta, `Test` nesnesi döndüren bir `suite` metoduyla kurulur.
  - Kara kutu mantığı: beklenen değer ile elde edilen sonuç karşılaştırılır. `Assert` sınıfının statik metotları: `assertEquals(beklenen, sonuç)`, `assertNotEquals`, `assertTrue`, `assertFalse`, `assertNull`, `assertNotNull`.
  - Test sınıflarını ayrı bir pakette tutmak daha mantıklıdır; en azından jar oluştururken test sınıfları dışarıda bırakılır.
- JUnit 4.x gelişmeleri (ekrandan, 00:40:00)
  - Geriye doğru uyumluluk korunur; annotation desteği gelmiştir (büyük/küçük harf duyarlı). `TestCase`'ten kalıtım gerekmez, şu eklemeler yapılır:

```
import static org.junit.Assert.*;
import org.junit.*;
```

  - Test metotlarının adını `test` ile başlatmak gerekmez; başlarına `@Test` konur. `setUp` yerine (adı ne olursa olsun) `@Before` işaretli metot kullanılır.

```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }
```

  - Beklenen bir istisnanın (exception) gerçekten fırlatılıp fırlatılmadığı sınanabilir. Checked exception ise metot `throws` ile bildirmek zorundadır. 3.x'te istisna program akışını keserdi; 4.x'te test başarılı ya da başarısız olsa da diğer testler sürer.

```
@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
```

```
}
```

- Kaynak ve video kiralama örneği (ekrandan, 00:50:00)
  - Konunun kaynağı Martin Fowler'ın *Refactoring* kitabıdır; hoca kitaptaki kusur ve düzenlemelerin yaklaşık yarısını seçmiştir. Adlar İngilizce özgün biçimleriyle kullanılır.
  - Senaryo: `Customer` sıfır ya da daha çok `Rental` (kira kaydı) tutar; her `Rental` bir `Movie` ile ilişkilidir; her filmin bir fiyat kodu vardır (yeni çıkan, normal, çocuk filmi). `Customer.statement()` kiralanan filmlerin adlarını, sürelerini, borcu, kazanılan bonus puanı ve toplamları içeren makbuzu ekrana yazmak yerine `String` olarak döndürür, böylece beklenen metinle karşılaştırılarak test edilebilir.
  - Testler: `TestCustomer` (5 test durumu: kirasız makbuz, uzun ve kısa süreli kiralama, yeni çıkan filmlerde fazladan bonus vb.) ve `TestMovie` (1 test durumu); `TestAll` suite'i toplam 6 testi çalıştırır, hepsi başarılı (yeşil çubuk).
  - Kod örneklerinde alt çizgiyle başlayan değişkenler (`_name`, `_rentals`) üye alanlardır.
- Duplicated Code (yinelenen kod) ve Extract Method (ekrandan, 01:10:00, 01:20:00)
  - Aynı ya da çok benzer kodun birden çok yerde tekrarlanmasıdır; her değişiklik bütün kopyalara ayrı ayrı yapılmalıdır, sonuç bir bakım kabusudur.
  - Extract Method: yinelenen ya da ayrı bir iş yapan kod parçası, işini adından anlatan yeni bir metoda çıkarılır; adı uzun olabilir, yorum ihtiyacını ortadan kaldırmalıdır. "Böl, parçala, yönet" ilkesinin uygulamasıdır.
  - `printOwing` örneği: banner basma, borç hesaplama ve ayrıntı yazma üç ayrı iştir; `printBanner()`, `getOutstanding()`, `printDetails(double)` metotlarına çıkarılır; `printOwing` iki satıra iner: `printBanner();` `printDetails(getOutstanding());`. Gereksiz geçici değişken kullanılmaz; `while` döngüsü for-each'e çevrilir.
- Long Method (aşırı uzun metot)
  - Metot uzadıkça anlaşılması zorlaşır, hata olasılığı ve gereksinim değişikliğinden etkilenme olasılığı artar, uyum düşer (birbirinden uzak işler aynı metotta toplanır).
  - Çözüm: Extract Method ile her alt iş ayrı metoda çıkarılır. Ayrıca Decompose Conditional, Replace Temp with Query ve Replace Method with Method Object.
  - Parça sayısı artar ama IDE metotlar arasında gezinmeyi kolaylaştırır. Modern derleyiciler metot çağrısı ek yükünü büyük ölçüde optimize eder; darboğaz genellikle başka yerdedir (ör. ağ haberleşmesi). Başarım gerçekten önemli ölçüde düşerse küçük adım geri alınır.
- Yinelenen kod için diğer düzenlemeler
  - Pull Up Method (ekrandan, 01:30:00): kardeş sınıflarda yinelenen metot üst sınıfa çekilir. Örnek: `Salesman` ve `Engineer`'daki `getName`, `Employee`'ye taşınır; küçük farklar `override` ile (gerekirse `super` çağrılarak) karşılanır. Pull up üst sınıfa, pull down alt sınıfa taşımadır.
  - Form Template Method: yinelenen kod kısmi ise (ör. `RegularCustomer` ve `PreferredCustomer`'daki `chargeFor(start, end)` metotlarının bir kısmı aynı, bir kısmı farklıysa) metodu doğrudan yukarı çekmek gereksiz kalıtım doğurur; Template Method kalıbı kurulur. Hiçbir şey yapılamıyorsa en azından metot üst sınıfa soyut olarak konur.
  - Yinelenen kod ilgisiz sınıflardaysa ortak kod uygun bir sınıfa çıkarılır, diğerleri oradan çağırır. Bağlaşım biraz artar ama yüzlerce ilişkiden oluşan bir sistemde bu artış küçük, yinelenen kodun bakım maliyeti ise büyüktür.
- Decompose Conditional (ekrandan, 01:40:00)
  - Long Method'un ana nedeni karmaşık if-else zinciri, switch-case ya da iç içe döngülerse uygulanır. Koşul ve dallardaki hesaplar adı anlamlı metotlara çıkarılır.

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
    date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
```

```
        return quantity * _summerRate;
    }
    private double winterCharge(int quantity) {
        return quantity * _winterRate + _winterServiceCharge;
    }
```

  - Hoca koşulu anlamak için zaman ekseni çizdi: yaz başlangıcından önce ya da yaz sonundan sonra olan dönem kış tarifesidir. Yalnız koşulu ya da yalnız hesapları düzeltmek "hibrit" kalırdı; hepsi ayrıldı.

- Long Parameter List (uzun parametre listesi) (ekrandan, 01:56:25)
  - Bir metodun çok parametre alması ve aynı parametre grubunun başka metotlarda da birlikte geçmesi, bu parametrelerin bir varlığın durum bilgisini oluşturduğunun göstergesidir.
  - Introduce Parameter Object: birlikte geçen parametreler yeni bir sınıfta üye alan olarak toplanır. `Customer`'ın `amountInvoicedIn`, `amountReceivedIn`, `amountOverdueIn(start: Date, end: Date)` metotları `(DateRange)` alır. `DateRange` yalnız bir kayıt yapısı kalmaz; `isWithinRange(Date)` gibi metotlar da ona taşınır (decompose conditional ya da extract method). Sorumluluk, onu yerine getirecek veriye sahip varlığa konur.
  - Preserve Whole Object: bir nesnenin üyelerini ayrı parametreler olarak göndermek yerine nesnenin kendisi gönderilir.

```
/*** ÖNCE: ***/
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);
/*** SONRA: ***/
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

  - Zayıf yönler: gönderilen nesnenin sınıfı ile metodun sahibi sınıf arasında yeni bir bağımlılık (kullanma ilişkisi) oluşabilir. Ayrıca metodun imzası değişir; başka sınıflar kullanıyorsa geriye doğru uyumluluk bozulur, bu durumda tam bir refactoring sayılmaz.

- Replace Temp with Query (ekrandan, 02:10:00, 02:14:40)
  - Bir ifadeyi geçici değişkende saklamak yerine ifadeden sorgu metotları çıkarılır.

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}
/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

  - Geçici değişkenler kodu uzatır, karmaşıklaştırır ve kopyala-yapıştırı teşvik eder.
  - Bedel: `basePrice()` iki kez hesaplanır. Döngü içinde (`printRecipt` örneğinde `thisAmount` yerine iki kez `item.getCharge()`) çağrı sayısı ikiye katlanır. Hesap uzak bir serviste ya da yavaşsa ve başarım hissedilir biçimde düşüyorsa yapılmaz; aksi hâlde yapılır. Kümülatif toplam (`totalAmount`) ise kaldırılamaz.

- Replace Method with Method Object (ekrandan, 02:20:00)
  - Metot o kadar uzun ve geçici değişkenleri öyle karışık kullanıyor ki Extract Method uygulanamıyorsa: metot bütünüyle yeni bir sınıfa taşınır, yerel değişkenler o sınıfın üye alanları olur. Sonra bilinen düzenlemeler daha rahat uygulanır.
  - Örnek: `Order.price()` içindeki `primaryBasePrice`, `secondaryBasePrice`, `tertiaryBasePrice` ve uzun hesap `PriceCalculator` sınıfına taşınır; `Order.price()` yalnız `return new PriceCalculator(this).compute()` olur (delegate metot).

- Large Class (aşırı büyük sınıf)
  - Rakamsal tek bir eşik yoktur; sınıf diğerlerine göre çok büyükse (dosya boyutu, üye alan ve metot sayısı) kusurdur. Görselleştirme ve kod analiz araçları (ör. Sonar) yardımcı olur.
  - Büyük sınıf çok fazla ve birbiriyle ilgisiz sorumluluk taşır, uyumu düşüktür.
  - Çözümler: Extract Class, Extract Subclass, Duplicate Observed Data.
  - Extract Class (ekrandan, 02:28:00): birbiriyle ilgisiz iki iş yapan sınıf ikiye ayrılır. `Person {name, officeAreaCode, officeNumber; getTelephoneNumber}` → `Person {name; getTelephoneNumber}` ──officeTelephone, 1──► `TelephoneNumber {areaCode, number; getTelephoneNumber}`.
  - Extract Subclass (ekrandan, 02:30:00): bazı üyeler her zaman kullanılmıyorsa onlardan alt sınıf oluşturulur. `JobItem {getTotalPrice, getUnitPrice, getEmployee}` → `JobItem {getTotalPrice, getUnitPrice}` ◁── `LaborItem {getUnitPrice, getEmployee}` (belirli bir uzman çalışana yaptırılan işler).
  - Duplicate Observed Data (ekrandan, 02:34:30): uygulama alanına ait veri yalnız bir GUI sınıfında saklanmamalıdır; veri iş mantığı katmanında yeni bir sınıfa taşınır ve GUI nesnesi ile bu nesne arasında Observer kurulur. `IntervalWindow` (startField, endField, lengthField: TextField; `StartField_FocusLost` vb.) Observer'ı gerçekler; `Interval` (start, end, length: String; `calculateLength`, `calculateEnd`) Observable'dır. Böylece katmanlı mimari korunur.
- Divergent Change ve Shotgun Surgery
  - Ortak kök neden: bir değişikliğin çok sayıda yerde yapılmasını gerektirmesi.
  - Değişiklikler aynı sınıfın çok sayıda metodunda toplanıyorsa Divergent Change; çok sayıda sınıfın metotlarına dağılıyorsa Shotgun Surgery (saçmalı tüfek benzetmesi).
  - Çözümler: Extract Class, Move Method, Move Field. Move Method: bir metot başka sınıftan çok daha sık çağrılıyorsa oraya taşınır; eski yerde yalnız çağrıyı ileten tek satırlık bir metot kalabilir.
- Primitive Obsession (ilkel türlerin aşırı kullanımı)
  - Kökeni yapısal programlama alışkanlıklarıdır ("geriye ket vurma": eski bilginin yenisinin önüne geçmesi).
  - Düzenlemeler: Replace Data Value with Object, Replace Type Code with Class, Replace Type Code with Subclass, Replace Type Code with State/Strategy, Replace Array with Object; ayrıca Extract Class ve Introduce Parameter Object.
  - Replace Data Value with Object (ekrandan, 02:50:00): `Order {customer: String}` → `Order` ◆──1──► `Customer {name: String}`. Ad yazımı tutarsızlıkları ve aynı adlı birden çok müşteri sorun olur; müşteriyle ilgili işlemler çoğaldıkça sınıf ayrılmazsa `Order` aşırı büyür.
  - Replace Type Code with Class: kan grubu gibi sabit kodlar (O=0, A=1, B=2, AB=3) yerine ayrı bir `BloodGroup` sınıfı ya da dil destekliyorsa enum kullanılır; eski (legacy) kodla uyumluluk gerekiyorsa sınıf tercih edilir.
  - Replace Type Code with Subclasses (ekrandan, 02:57:20): `Employee`'deki `ENGINEER`, `SALESMAN`, `MANAGER` sabitleri ve `work()` içindeki switch-case yerine soyut `Employee` ve `work()`'ü gerçekleyen `Engineer`, `Salesman` alt sınıfları. Önceki hâlde `default` varsa `Employee` soyut yapılmaz.
  - Replace Type Code with State/Strategy (ekrandan, 03:00:00): `Employee` ──1──► `EmployeeType` ◁── `Engineer, Salesman`. Bir nesne çalışma sırasında sık sık tür değiştiriyorsa (mühendisten yöneticiye terfi ve geri dönüş gibi) Strategy daha doğrudur.
  - Replace Array with Object: farklı türde bilgileri iki boyutlu dizide ya da paralel dizilerde (takım adı, puan, averaj) tutmak yerine `Takım/Skor` sınıfı tanımlanır.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod kusuru hata değildir
  - Hoca "kusur" kelimesini bilerek seçtiğini ve bunun hata (bug) olmadığını birkaç kez vurguladı; kusur, kodun bakımını zorlaştıran tasarım sorunudur.
- Kusur ve refactoring adları İngilizce özgün biçimiyle bilinmelidir
  - Hoca, kitaptaki özgün code smell ve refactoring adlarını sınavlarda kullanmayı beklediğini söyledi.
- Mediator ile Observer'ın farkı
  - Şemaları benzer, amaçları farklıdır: Observer değişiklikten haberdar etmek içindir;

Mediator eşten eşe ilişkileri tek bir arabulucuda toplar.
- Refactoring küçük adımlarla ve sınamayla yapılır
  - Her değişiklikten sonra birim testler çalıştırılmalıdır; test etmenin mazereti yoktur. Test etmekte zorlanılıyorsa tasarım büyük olasılıkla yanlıştır.
- Başarım tartışması
  - Küçük metotların çalışma anı ek yükü modern derleyicilerle hissedilmez; tartışma "bu refactoring hissedilir bir darboğaz yaratıyor mu" ekseninde yapılmalıdır. Yaratıyorsa adım geri alınır; darboğaz başka yerdeyse refactoring yapılmalıdır.
- Gereksiz geçici değişken kullanılmamalı
  - Geçici değişkenler kodu uzatır, karmaşıklaştırır ve kopyala-yapıştırı teşvik eder.
- Metot adları yaptığı işi anlatmalı
  - Ad uzun olabilir; amaç yorum satırına ihtiyaç bırakmamaktır.
- if-else ya da switch-case zinciri çok biçimliliğe ihtiyacın işaretidir
  - Tür kodu üzerinden dallanan kod Replace Type Code with Subclass (ya da tür değişebiliyorsa State/Strategy) ile düzeltilmelidir.

Kısa Tekrar Notları

- Mediator: meslektaşlar birbirine değil arabulucuya mesaj gönderir; $m \times n$ ilişki merkeze toplanır. Örnek: `FontDialogDirector`, `Widget.Changed()` → `director->WidgetChanged(this)`.
- Mediator karmaşıklığı yok etmez, merkeze taşır; etkileşimler iyi tanımlı ve kararlıysa kurulur.
- Refactoring: dış davranışı değiştirmeden iç yapıyı iyileştirmek. Küçük adımlar + her adımda test.
- JUnit 3: `TestCase`'ten kalıtım, `test` önekli metotlar, `setUp`, `suite`, `assertEquals/True/False/Null/NotN`
- JUnit 4: `@Test`, `@Before`, `import static org.junit.Assert.*`, `@Test(expected=...)`.
- Duplicated Code → Extract Method, Pull Up Method, Form Template Method.
- Long Method → Extract Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object.
- Long Parameter List → Introduce Parameter Object (`DateRange`), Preserve Whole Object (zayıf yön: bağımlılık ve imza değişikliği).
- Large Class → Extract Class (`TelephoneNumber`), Extract Subclass (`LaborItem`), Duplicate Observed Data (`Interval` + Observer).
- Divergent Change (tek sınıfta çok değişiklik) ve Shotgun Surgery (çok sınıfa dağılmış değişiklik) → Extract Class, Move Method, Move Field.
- Primitive Obsession → Replace Data Value with Object, Replace Type Code with Class/Subclass/State-Strategy, Replace Array with Object.
- Tür sık değişiyorsa (terfi) Subclass yerine Strategy.

Detaylı Açıklamalar

- Mediator kalıbında font penceresi örneği, kalıbın neden doğduğunu iyi gösterir. Pencerede bir liste, bir metin kutusu ve kalınlık, italik, punto seçicileri vardır; hepsindeki her değişiklik önizleme alanına yansımalıdır. Bileşenler önizleme alanını doğrudan değiştirseydi, her GUI bileşeni başka bir bileşenin metotlarını bilmek zorunda kalır ve aralarında çok fazla benzer kod oluşurdu. Bunun yerine her bileşen yalnız "değiştim" der (`Changed()` içinde `director->WidgetChanged(this)`), `FontDialogDirector` hangi bileşenin değiştiğine bakar, gerekirse diğer bileşenleri sorgular (`GetSelection()`) ve önizlemeyi ya da diğer bileşenleri günceller (`SetText()`). Bu kısım Observer'a benzer; fark, tek bir arabulucunun bütün eşler arasındaki trafiği yönetmesidir. Merkezi topolojinin dezavantajı merkezin yükü, eşten eşe topolojinin dezavantajı ise her bileşene karmaşık bağlantı kodu düşmesidir; probleme hangisi uygunsa o seçilir. Hoca kalıbı "çok fazla şey vaat etmeyen" bir kalıp olarak değerlendirdi ve bu yüzden ayrı bir uygulama örneği yapmadı.
- Refactoring'in neden küçük adımlarla yapıldığı şöyle açıklandı: yazılımın 2.0 sürümünden 2.2'ye geçerken bütünüyle yeni bir tasarım yapılırsa bu aylar sürebilir; o sürede sahada yeni gereksinimler çıkar ve yeni tasarımın da yeterince esnek olmadığı anlaşılabilir. Küçük dokunuşlarla düzeltmek daha kolaydır. Her küçük adım, sistemin başka bir yerini bozmuş olabileceği için testle doğrulanır. Test için tüm sınıfların bitmesini bekle-

mek gerekmez: geliştirme aracı sınıf şemasından iskelet kod üretebilir; henüz yazılmamış sınıfların yerine her zaman başarılı dönen vekil sınıflar konur, gerçek kod tamamlanınca tümleştirme testine geçilir.

- JUnit'te bir test sınıfı, uygulama sınıfıyla sahiplik ilişkisi kurmak zorunda değildir; en azından kullanma ilişkisi vardır. `setUp` her test metodundan önce çalıştığı için her test temiz bir ortamda başlar. Video kiralama örneğinde `TestCustomer`'ın `setUp` metodu müşteriyi ve farklı türdeki filmleri ilklendirir ama kiralama ilişkilerini kurmaz, çünkü testlerden biri hiç kiralama yapılmamışken basılan makbuzu sınar. Uzun ve kısa süreli kiralama ve yeni çıkan filmlerde fazladan bonus ayrı test durumlarıdır. Her testin sonunda beklenen makbuz metni ile `statement()`'ın döndürdüğü metin `assertEquals` ile karşılaştırılır. Hoca refactoring ve test kodunu ayrı klasörlerde (`refactoring/fowler_00`, `testing/fowler_00` gibi) ama aynı paket adıyla tuttuğunu, Eclipse'te bunun yapılandırılabildiğini gösterdi.
- Extract Method en temel düzenlemedir ve birçok kusurun çözümünde alt adım olarak kullanılır. `printOwing` örneğinde önceki metot bir yorumla "print banner", bir yorumla "calculate outstanding", bir yorumla "print details" bölümlerine ayrılmıştı. Yorumla açıklama ihtiyacı, o parçanın ayrı bir metot olması gerektiğinin işaretidir. Sonraki hâlde `printOwing` metodu ne yaptığını iki satırda özetler; ayrıntı gerekirse ilgili metoda inilir. Hoca `printDetails(getOutstanding())` biçiminde iç içe çağrı yaparak ara bir geçici değişken tanımlamadığını özellikle belirtti; `getOutstanding` içindeki `outstanding` ise kümülatif toplam olduğu için kaçınılmazdır.
- Pull Up Method, Form Template Method ve ilgisiz sınıflara çıkarma arasındaki seçim şöyle özetlendi: çıkarılan metot aynı sınıfta kalıyorsa Extract Method, kalıtım hiyerarşisinde üst sınıfa çıkıyorsa Pull Up Method, doğrudan yukarı çekmek uygun değilse (yinelenen kod kısmi ise) Form Template Method yapılır. Template Method sonucunda bazı alt sınıflarda boş gövdeli metot kalsa bile bakım daha kolaydır.
- Decompose Conditional örneğinde hoca, `date.before(SUMMER_START) || date.after(SUMMER_END)` koşulunun doğru olup olmadığını anlamak için bile kâğıt kalem gerektiğini gösterdi. Koşulun anlamı "yaz değilse" olduğu için `notSummer(date)` adlı `private` bir metoda çıkarıldı. Başka sınıflar da benzer bir tarih denetimine ihtiyaç duyarsa metot `public static` yapılabilir. Karmaşıklık yok edilmedi ama ayrı bir metoda yerleştirildi; sonradan bakan programcı yalnız o metodu ya da onun testini inceleyebilir.
- Replace Temp with Query için hocanın değerlendirmesi açıktır: bir çarpma işleminin ikinci kez yapılması ya da bir metot çağrısının ek yükü günümüz bilgisayarlarında hissedilmez. Ancak hesaplar ayrı makinelerde, yüksek gecikmeli bir ağ üzerinden çağrılan servislerse (hocanın öğrencilerden duyduğuna göre bankacılık sistemlerinde sık görülür) başarım hissedilir biçimde düşebilir; o zaman yapılmaz. Döngü içindeki örnekte `thisAmount` değişkeni bir kez atanıp hiç değiştirilmeden kullanıldığı için her yerde `item.getCharge()` ile değiştirilebilir; çağrı sayısı öğe sayısının iki katına çıkar. Birkaç yüz öğede fark etmez, milyonlarca öğede ya da `getCharge` başka bilgi sistemlerinden veri çekiyorsa fark edebilir. Testlerin süresi ölçülerek karar verilir; hoca örneğin toplamda %5'lik bir düşüşü önemli sayılabilecek bir rakam olarak andı.
- Replace Method with Method Object, Extract Method'un yetmediği durumlar içindir: metotta çok sayıda geçici değişken vardır, bunlar if-else dallarında değiştirilip başka yerlerde değişmiş hâlleriyle kullanılır. Metodun tamamı yeni bir sınıfa taşınınca yerel değişkenler üye alana dönüşür ve artık metodun parçaları parametre aktarmadan ayrı metotlara çıkarılabilir. Yeni sınıfın adı taşınan metodun adına benzer seçilir (`price` → `PriceCalculator`); duruma göre Singleton da olabilir.
- Duplicate Observed Data, katmanlı mimariyi korumak içindir. Veri yalnız `IntervalWindow`'un metin alanlarında tutulursa ve hesaplar (`calculateLength`, `calculateEnd`) pencere sınıfında yapılırsa sınıf hem sunum hem iş mantığı katmanına bulaşır. Çözümde veri `Interval` sınıfında çoğaltılır; GUI olayları gerçekleştikçe `Interval`'ın set metotları çağrılır ve asıl işi `Interval` yapar. Çok kullanıcılı bir sistemde `Interval` başka bir nedenle değişirse pencerenin haberdar olması gerektiği için pencere Observer, `Interval` Observable olur. Daha masraflıdır ama hoca bu düzenlemeyi kesinlikle tercih ettiğini söyledi.
- Replace Type Code with Subclass ile State/Strategy arasındaki seçim, hocanın bir soruya verdiği cevapta netleşti: türler iyi tanımlıysa ve bir nesne türler arasında dolaşmayacaksa alt sınıf çözümü yeterlidir. Ama bir çalışan zamanla mühendisten yöneticiye terfi edebiliyor, satış görevine geçebiliyor ya da yöneticilikten geri alınabiliyorsa alt sınıf çözümünde her seferinde ortak bilgilerle yeni bir nesne oluşturup eskisini silmek gerekir. Bu sık oluyorsa çalışana bir `EmployeeType` stratejisi atamak daha doğrudur.

Ders sonrası not: Replace Type Code with Subclasses slaytındaki `public abstract void work(){};` satırı Java'da derlenmez; soyut metodun gövdesi olamaz. Doğrusu `public abstract void work();` biçimindedir. Aynı slayttaki `extends Employe` de `extends Employee` olmalıdır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.